I want to demo an application for my final year project. Find attached the document. I plan on using Flutter for maximum compatibility with most main OS platforms

I want you to focus on chapter 3 mainly, coz that's where my implementation designs are. Then look for all possible libraries that we might need for the project.

So what I want you to do now is just respond with everything you understand about the project I want to do.

Hybrid-Decentralized-NetworkDigitalCommunicationnCommerceinLow.pdf

Here's what I understand about the project you want to build.

1. Core idea of the project

You want to build a proof-of-concept application for low-connectivity environments, especially places like Landmark University hostels where internet is unstable, slow, or unavailable.

The system is not just a normal chat app or file-sharing app. It is a hybrid decentralized network platform that lets nearby devices continue to communicate and exchange value even when the internet is down, and then synchronize with the internet later whenever any device regains access.

So the project is basically:

- offline-first
- peer-to-peer for local communication
- delay-tolerant
- partially decentralized locally
- centrally reconciled when internet becomes available

The name used in Chapter 3 is OffLiMU.

2. What problem the project is solving

Your project is trying to solve the dependence of modern apps on constant internet access.

Right now, most apps fail completely when the connection drops because they depend on centralized servers in real time. Your proposed system avoids that by allowing devices in the same environment to still:

- chat
- share files and media
- browse cached web content
- perform token-based local transactions
- support simple commerce workflows
- sync later through any node that temporarily has internet

So the system is designed for “graceful degradation” instead of total failure.

3. The type of system you want to demo

This is not just one app screen with one feature. It is a multi-service platform with several modules built on one shared networking backbone.

The user-facing features are:

- Chat and messaging
- File and media sharing
- Offline digital wallet / token transactions
- Local e-commerce / trading
- Offline web search / cached web browsing
- Background synchronization with server when internet returns

So what you really want to demo is a mini offline digital ecosystem.

4. The environment the system is meant for

Your case study is Landmark University student hostels.

That means the deployment assumptions are:

- many users are physically close enough for local network interactions
- internet is intermittent

- devices may move around
- some devices may temporarily have internet
- not every device is online at the same time
- the system must tolerate disconnections and delayed delivery

So the project is designed more like a campus/community offline mesh-assisted ecosystem than a classic online app.

5. The architectural philosophy

The most important design philosophy in Chapter 3 is:

- local-first
- network-agnostic app layer
- store-carry-forward networking
- eventual consistency
- opportunistic synchronization
- modular architecture

That means the app should still work from local state even if network state changes.

The UI should not depend directly on “am I online?” to function. Instead, users perform actions locally, and the system wraps those actions as events/bundles, stores them, and forwards them whenever possible.

6. Main architectural model

The system has 3 conceptual parts:

a. OffLiMU Nodes
These are ordinary participating devices like phones or laptops.

Each node can:

- run the app locally
- create and receive messages/files/transactions
- store data persistently

- forward data for others
- participate in opportunistic peer-to-peer communication

b. Gateway Nodes
A gateway is not a permanently special device.
It is just a normal node that temporarily has internet.

When internet is detected, that node can:

- upload pending data to the server
- download updates/content from the server
- synchronize payments and indexes
- act as a bridge between local offline mesh and the online world

When connectivity disappears, it goes back to normal offline node behavior.

c. Central Server
This is not for real-time control of the local network.
It exists for:

- reconciliation
- global token ledger
- scraping/fetching web content
- content aggregation
- synchronization hub functions

So your system is hybrid:
locally decentralized, globally assisted by a central authority.

7. Connectivity modes

The app is supposed to adapt to different connectivity conditions:

- Offline DTN mode:
  no internet assumed, bundles stored locally
- LAN DTN mode:
  one or more peers reachable on local network

- Gateway mode:
  internet is available, node can talk to central server
- Sync mode:
  deferred reconciliation and upload/download happens

This is important because your system behavior changes based on context, but the app experience should remain usable.

8. Internal node architecture

Chapter 3 describes a layered node architecture.

A. Application Layer
This is the user-facing module layer.

Modules include:

- messaging
- files
- wallet
- commerce
- offline search/web

These modules should:

- work with local data
- not open sockets directly
- not manage low-level networking themselves
- emit high-level intents like SendMessage, StoreFile, MakePayment, SearchContent

So this layer is isolated from networking complexity.

B. Application API Layer
This acts as the bridge between app features and the network core.

Its job is to:

- convert user intents into DTN bundles

- add metadata like source, destination, TTL, CID
- route received bundles back to the right feature module

So this is effectively your protocol adapter between app logic and transport logic.

C. DTN Core Layer
This is the real heart of the system.

It implements store-carry-forward behavior.

Main subcomponents:

1. Ingress Handler

- accepts incoming bundles from local app or remote peers
- validates structure
- checks signature
- checks TTL
- deduplicates
- admits valid bundles to storage

2. Bundle Store

- persistent storage for pending/delivered/acknowledged bundles
- survives app restarts and reboots
- acts as local DTN queue

3. Contact Manager

- discovers nearby peers
- tracks availability and contact info
- stores encounter history

4. Routing Policy Engine

- scores which peer is best to forward which bundle to
- supports routing strategies like PRoPHET or MaxProp

- separated from app logic

5. Forwarder

- checks pending bundles
- checks available contacts
- chooses what to send
- performs negotiation
- triggers transmission
- coordinates large file transfer via data layer

This means your app is not simple socket messaging. It is a proper delay-tolerant forwarding system.

D. Data Layer
This handles heavy content like:

- images
- documents
- media
- web snapshots

Instead of stuffing large binaries directly into bundles, the design uses content-addressed storage.

Main idea:

- file is stored in IPFS-like structure
- file gets chunked
- file gets a CID
- lightweight DTN bundle references the CID
- actual heavy transfer happens separately and opportunistically

This is smart because it reduces routing overhead and supports resumable transfers.

E. Transport & Connectivity Layer

This is the actual movement of bytes.

It supports:

- peer-to-peer local communication over LAN / Wi-Fi / maybe ad hoc
- secure HTTP/HTTPS synchronization to server when online

So it treats the server almost like a high-quality peer rather than redesigning the whole protocol.

9. Bundle-based communication model

A huge part of your design is that everything becomes a bundle.

That includes:

- chat messages
- file references
- payment events
- search requests
- index updates
- acknowledgements
- server confirmations

A typical bundle contains things like:

- bundle ID
- app ID
- type
- source
- destination
- timestamps
- TTL
- hop count
- priority

- payload or CID
- signature

So the system is event-driven and bundle-driven.

10. File/media transfer design

Your file transfer design is not simple direct send.

The intended workflow is:

- user selects file
- app API stores file in content-addressed storage
- data layer returns CID
- a DTN bundle containing CID is created
- routing system forwards the metadata bundle
- when a peer is authorized to fetch, the data layer performs chunk/block exchange
- transfer can pause and resume across intermittent contacts

So heavy content transfer is separated from control/routing logic.

That is one of the strongest technical ideas in your Chapter 3.

11. Payment and incentive mechanism

You are not implementing real banking.

Instead, you are designing a system-defined token economy for experimentation.

Key points:

- tokens are internal only
- they have no external monetary value
- nodes can spend tokens offline
- transactions are cryptographically signed
- local wallets are append-only logs
- balance is derived from valid incoming/outgoing events

- final validation happens only at the server during reconciliation

When offline:

- a spend event is created
- signed by sender
- propagated through DTN

When a gateway reaches internet:

- spend events are uploaded
- server checks signature
- server detects double-spend
- server reconciles ordering
- valid ones are committed
- invalid/conflicting ones are rejected
- result is broadcast back as confirmation/rejection bundles

You also want incentives:

- nodes get rewarded for forwarding data
- nodes get rewarded for acting as gateways
- server issues reward events based on logs and acknowledgements

So your payment system is really:
offline provisional transactions + centralized eventual settlement.

12. E-commerce meaning in this project

The commerce part is not a full-scale marketplace like Jumia or Amazon.

It is more like:

- local product listings
- cached catalog browsing
- placing orders locally

- provisional payment using internal tokens
- later reconciliation when online

So the commerce demo would probably show:

- a buyer browsing local listings
- making an order
- paying with internal token
- merchant receiving pending confirmation
- final server-backed settlement later

13. Offline web access and search

Another major feature is offline web search and cached browsing.

This works through three actors:

a. Server

- crawls/fetches web content from the internet
- converts pages into static snapshots
- stores them
- produces metadata/index entries

b. Gateway

- downloads new snapshots/indexes from server
- injects them into local network
- spreads update bundles

c. Offline Node

- keeps local search index
- lets user search locally
- loads cached content if available
- if not available, can issue an async search request to be resolved later

So this is not live Google search.

It is more like:

- local searchable cache
- plus deferred fetch request if a gateway later gets internet

That fits your low-connectivity use case well.

14. Acknowledgement and lifecycle management

Your design also includes ACK bundles.

These are important because in DTN, sending something does not mean it was delivered.

ACKs are used to:

- confirm successful delivery
- allow garbage collection
- remove redundant bundle copies
- support incentive accounting

So bundle lifecycle is something like:

- created
- stored
- forwarded
- delivered
- acknowledged
- eventually deleted

If TTL expires before acknowledgment:

- bundle is discarded

This is important because storage and congestion control matter in disconnected systems.

15. Role of the central server

The server is lightweight but essential.

It is responsible for things local nodes should not do alone:

- authoritative token ledger
- payment reconciliation
- double-spend detection
- web crawling and snapshot generation
- synchronization hub for geographically separated mesh groups
- content mirroring/pinning

So although the system is “decentralized,” it is not fully trustless or fully serverless.

It is better described as:
hybrid decentralized with centralized reconciliation and content aggregation.

16. Planned implementation style in the document

The document originally proposes Go-based implementation pieces like:

- Go backend/services
- WebView UI
- BoltDB
- mDNS/Zeroconf
- raw TCP
- IPFS/Kubo
- crypto libraries
- server scraper

But now you want to use Flutter for maximum compatibility.

That means, from my understanding, you want to preserve the Chapter 3 architecture while changing the implementation technology for the client app.

So in Flutter terms, your plan is likely to keep the architecture but rebuild the frontend/client around Flutter instead of the original WebView/Go-heavy prototype design.

17. How Flutter fits your project

Flutter makes sense for your goal because you want broad platform support.

This likely means you want one codebase targeting:

- Android
- Windows
- possibly Linux
- possibly macOS
- maybe iOS too, though local networking restrictions may complicate some features

But using Flutter does not remove the architectural challenges.
You still need solutions for:

- local peer discovery
- background sync
- local database
- file storage
- cryptographic signing
- content addressing
- LAN/Wi-Fi/Bluetooth communication
- server sync APIs

So Flutter can handle the app/UI/shared logic very well, but some lower-level networking or platform-specific functions may still require:

- plugins
- platform channels
- native code bridges
- companion services in Go/Rust/Kotlin/Swift if needed

18. What the demo likely needs to prove

From Chapter 3, I think the demo should prove these ideas:

1. A node can work offline

- app opens
- local state exists
- user can use features without internet

2. Two nearby nodes can communicate directly

- send message
- send file
- receive without internet

3. Data is persistent

- after app restart, pending messages/files still exist

4. Gateway behavior works

- when one device gets internet, it syncs pending data

5. Payment flow works

- offline spend event created
- later reconciled online
- confirmation or rejection comes back

6. Offline search works

- local cached content can be searched and opened
- missing content can be requested asynchronously

7. ACK lifecycle works

- delivered bundles become acknowledged
- storage queue shrinks appropriately

19. What is intentionally out of scope

Your project is a proof-of-concept, so several things are explicitly not included:

- full blockchain integration
- real money integration
- banking infrastructure
- large-scale deployment
- specialized hardware
- global-scale routing
- full adversarial security model
- full sustainability/governance model
- energy/battery optimization modeling
- complex distributed compute execution
- full iOS/Linux support in the document's original scope

So the project is about feasibility and architecture, not production-grade deployment.

20. Evaluation plan in Chapter 3

You also plan to evaluate the system using simulation and prototype deployment.

Simulation:

- use The ONE simulator
- model mobile nodes, gateways, server node
- compare routing protocols like:
  - PRoPHET
  - MaxProp
  - Epidemic

Metrics:

- delivery ratio
- average latency
- overhead ratio
- payment resolution time

- buffer occupancy

The simulation abstracts things like:

- crypto cost
- IPFS internals
- battery drain
- UI latency

So the evaluation is focused mostly on network behavior and DTN performance.

21. The kind of app architecture you are really building

In simple terms, your project is basically this:

A Flutter-based offline-first campus app platform where nearby devices can exchange messages, files, transaction events, and cached content directly, while any internet-connected device temporarily acts as a gateway to synchronize with a central server for reconciliation and broader content access.

22. My understanding of the most important technical pillars

If I boil the whole project down, these are the pillars:

- Flutter app for multi-platform UI
- Local persistent storage
- Peer discovery and peer-to-peer transport
- DTN bundle queue and forwarding logic
- Content-addressed file/media storage
- Tokenized offline transaction system
- Gateway-based sync to central server
- Offline searchable cached web content
- ACK and lifecycle management
- Server for reconciliation and web scraping

23. What I think your final demo should feel like

The strongest version of your demo would show something like this:

- Device A and Device B are offline
- A sends B a message
- A shares a file
- B receives both locally
- A pays B with internal tokens
- B sees pending receipt
- Users browse a local marketplace
- Users search already-cached web content
- Later Device C gets internet and becomes gateway
- Pending messages/logs/payments/content requests sync
- Server validates transaction
- Confirmation comes back into local network
- Missing web result becomes available later

That would clearly demonstrate your architecture.

24. One important observation

Your project is not really “just a Flutter app.”
It is a distributed systems project wrapped in an app.

The UI is only one part.
The real project is the coordination between:

- local storage
- routing
- discovery
- synchronization
- consistency
- offline event propagation
- content transfer

- eventual reconciliation

That is why Chapter 3 is the most important chapter: it defines the system behavior, not just screens.

25. In one sentence

What I understand is that you want to build a Flutter-based proof-of-concept hybrid decentralized offline-first platform called OffLiMU that enables messaging, file sharing, offline tokenized transactions, commerce, and cached web access in low-connectivity environments using DTN-style peer-to-peer communication, content-addressed storage, and opportunistic gateway synchronization with a central server.

If you want, next I can turn this understanding into either of these:

1. a clean Flutter-oriented system architecture
2. a full list of libraries/plugins you may need for each module
3. a practical MVP breakdown for what to build first
4. a mapping from your Chapter 3 design to Flutter implementation choices

Let's discuss about the clean Flutter-oriented system architecture & a full list of libraries/plugins you may need for each module

This is a meaty discussion, so let me break it into two major parts clearly.

Part 1: Clean Flutter-Oriented System Architecture

Overall Approach

The original Chapter 3 design was built around Go services with a WebView frontend. Flutter changes the implementation approach but the **architectural philosophy stays exactly the same**. What changes is how each layer is expressed in Flutter/Dart.

The key architectural pattern you should follow is **Clean Architecture** combined with a **Service Locator** or **Riverpod dependency injection** approach. This keeps your DTN core, data layer, and features completely separated.

High-Level Layered Diagram

text

```
┌──────────────────────────────────────────────┐
│              FLUTTER UI LAYER                │
│  (Messaging, Files, Wallet, Commerce,        │
│   Search, Settings)                          │
├──────────────────────────────────────────────┤
│           APPLICATION API LAYER              │
│  (BundleFactory, BundleDispatcher,           │
│   IntentHandler)                             │
├──────────────────────────────────────────────┤
│              DTN CORE LAYER                  │
│  (IngressHandler, BundleStore,               │
│   ContactManager, RoutingPolicyEngine,       │
│   Forwarder, ACKManager)                     │
├──────────────────┬───────────────────────────┤
│   DATA LAYER     │  TRANSPORT & CONNECTIVITY │
│  (ContentStore,  │  LAYER                    │
│   CID Manager,   │  (TCPTransport,           │
│   BlockExchange) │   HTTPTransport,          │
│                  │   PeerDiscovery)          │
├──────────────────┴───────────────────────────┤
│         SYNCHRONIZATION ENGINE               │
│  (GatewayManager, SyncQueue,                 │
│   ConnectivityWatcher)                       │
├──────────────────────────────────────────────┤
│         CRYPTOGRAPHIC IDENTITY LAYER         │
│  (KeyPairManager, Signer, Verifier)          │
└──────────────────────────────────────────────┘
         │
         │ HTTPS when internet available
         ▼
┌──────────────────────────────────────────────┐
│          OFFLUMU CENTRAL SERVER              │
│  (REST API, Token Ledger, Web Scraper,       │
│   Content Mirror)                            │
│  [Separate Go/Node.js service - not Flutter]│
└──────────────────────────────────────────────┘
```

Recommended Folder Structure

```text

lib/
│
├── main.dart
├── app.dart                    # App entry, DI setup, routing
│
├── core/
│   │
│   ├── identity/               # Cryptographic identity
│   │   ├── key_pair_manager.dart
│   │   ├── signer.dart
│   │   └── verifier.dart
│   │
│   ├── dtn/                    # DTN Core Layer
│   │   ├── models/
│   │   │   ├── bundle.dart
│   │   │   ├── ack_bundle.dart
│   │   │   └── bundle_status.dart
│   │   ├── bundle_store.dart
│   │   ├── ingress_handler.dart
│   │   ├── contact_manager.dart
│   │   ├── forwarder.dart
│   │   ├── ack_manager.dart
│   │   └── routing/
│   │       ├── routing_policy.dart    # abstract interface
│   │       ├── prophet_routing.dart
│   │       ├── maxprop_routing.dart
│   │       └── epidemic_routing.dart
│   │
│   ├── data_layer/             # Content-Addressed Storage
│   │   ├── content_store.dart
│   │   ├── cid_manager.dart
│   │   └── block_exchange.dart
│   │
│   ├── transport/              # Transport & Connectivity
│   │   ├── transport_interface.dart   # abstract
│   │   ├── tcp_transport.dart
│   │   ├── http_transport.dart
│   │   └── peer_discovery.dart
│   │
│   ├── sync/                   # Synchronization Engine
│   │   ├── sync_engine.dart
│   │   ├── gateway_manager.dart
│   │   ├── sync_queue.dart
│   │   └── connectivity_watcher.dart
│   │
│   └── crypto/                 # Crypto utilities
│       ├── hash_utils.dart
```

```
│       └── signature_utils.dart
│
├── app_api/                    # Application API Layer
│   ├── bundle_factory.dart
│   ├── bundle_dispatcher.dart
│   └── intent_handler.dart
│
├── features/                   # Feature Modules (UI + logic)
│   │
│   ├── messaging/
│   │   ├── data/
│   │   │   ├── message_repository.dart
│   │   │   └── message_bundle_adapter.dart
│   │   ├── domain/
│   │   │   └── message_model.dart
│   │   └── presentation/
│   │       ├── chat_screen.dart
│   │       ├── conversation_list_screen.dart
│   │       └── chat_provider.dart
│   │
│   ├── files/
│   │   ├── data/
│   │   │   ├── file_repository.dart
│   │   │   └── file_bundle_adapter.dart
│   │   ├── domain/
│   │   │   └── shared_file_model.dart
│   │   └── presentation/
│   │       ├── file_list_screen.dart
│   │       ├── file_send_screen.dart
│   │       └── files_provider.dart
│   │
│   ├── wallet/
│   │   ├── data/
│   │   │   ├── wallet_repository.dart
│   │   │   └── spend_event_adapter.dart
│   │   ├── domain/
│   │   │   ├── token_model.dart
│   │   │   └── spend_event_model.dart
│   │   └── presentation/
│   │       ├── wallet_screen.dart
│   │       ├── send_payment_screen.dart
│   │       └── wallet_provider.dart
│   │
│   ├── commerce/
│   │   ├── data/
│   │   │   ├── commerce_repository.dart
│   │   │   └── order_bundle_adapter.dart
│   │   ├── domain/
│   │   │   ├── product_model.dart
│   │   │   └── order_model.dart
│   │   └── presentation/
│   │       ├── marketplace_screen.dart
```

```
│   │       ├── product_detail_screen.dart
│   │       ├── checkout_screen.dart
│   │       └── commerce_provider.dart
│   │
│   └── search/
│       ├── data/
│       │   ├── search_index_repository.dart
│       │   └── index_update_adapter.dart
│       ├── domain/
│       │   └── web_snapshot_model.dart
│       └── presentation/
│           ├── search_screen.dart
│           ├── offline_browser_screen.dart
│           └── search_provider.dart
│
├── shared/                     # Shared UI widgets and utils
│   ├── widgets/
│   │   ├── connectivity_banner.dart
│   │   ├── bundle_status_indicator.dart
│   │   └── token_balance_chip.dart
│   └── utils/
│       ├── date_utils.dart
│       └── size_utils.dart
│
└── server/                     # Server API client (HTTP calls)
    ├── server_api_client.dart
    ├── models/
    │   ├── sync_upload_payload.dart
    │   └── sync_download_response.dart
    └── endpoints.dart
```

Key Architectural Decisions for Flutter

1. State Management: Riverpod

Use `flutter_riverpod` throughout. Each feature has its own providers. The DTN core and sync engine are exposed as global providers so any feature can listen to network/delivery events.

```dart
// example: connectivity state exposed globally
final connectivityProvider = StateNotifierProvider<ConnectivityNotifier,
ConnectivityState>(...);
```

```
// features consume it
final chatProvider = StateNotifierProvider<ChatNotifier, ChatState>(...);
```

2. Background Execution

This is the trickiest Flutter-specific challenge. Your DTN core needs to run in the background to:

- periodically scan for peers
- forward pending bundles
- watch for internet and activate gateway mode

You need a long-running background service, not just a one-shot task. Use `flutter_background_service` for this. On Android this spawns a foreground service. On Windows it runs in a background isolate.

```text
Background Service
├── Peer discovery loop (mDNS scanning)
├── Forwarder loop (every N seconds)
├── Connectivity watcher
└── Sync engine (activates when internet detected)
```

3. Isolate-Based DTN Core

Your DTN core logic is CPU and I/O intensive. Run it in a separate Dart Isolate so it does not block the UI thread. Use `flutter_isolate` or Dart's built-in `Isolate.spawn` with `SendPort` and `ReceivePort` for bidirectional communication.

```text
Main Isolate          DTN Isolate
(Flutter UI)  ←——→  (Core processing)
              ports
```

4. Database Strategy (Two Databases)

You need two separate databases:

- `Hive` or `Isar` for the DTN Bundle Store because it needs fast key-value access and crash safety. Bundle operations are high-frequency.

- `Drift` (type-safe SQLite ORM) for structured feature data like conversations, wallet history, product listings, orders, and search index.

5. Content-Addressed Storage: Lightweight CID

Full IPFS (Kubo) is not practical to embed in a Flutter app directly. Instead, implement a lightweight content-addressing system in Dart:

- chunk files into 256KB blocks
- SHA-256 hash each block
- combine hashes into a Merkle-like tree
- produce a root CID (base58/base32 encoded hash)
- store blocks in a local blocks directory under `path_provider`

This replicates the IPFS concept without running a full daemon. When a gateway syncs online, the server-side can run real IPFS/Kubo. Your Flutter app just needs content-addressable local storage.

6. Local Peer Transport: TCP over LAN

Use `dart:io` 's `ServerSocket` and `Socket` for local peer communication over Wi-Fi LAN. This works on both Android and Windows without native plugins.

Discovery uses mDNS via `bonsoir` to announce and discover nearby OffLiMU nodes on the local network.

Bundle transfer protocol over TCP:

```text
1. Connection established
2. Node A sends: HELLO + summary vector (list of bundle IDs it has)
3. Node B responds: REQUEST for bundles it does not have
4. Node A streams: requested bundles
5. Both nodes exchange ACKs for confirmed bundles
6. Connection closes
```

7. Routing Policy as a Plugin Interface

Define routing as an abstract Dart class. This lets you swap PRoPHET, MaxProp, or Epidemic at runtime.

```dart
abstract class RoutingPolicy {
  double score(Bundle bundle, PeerContact peer);
  void onContact(PeerContact peer);
  void onAckReceived(String bundleId);
}
```

8. Gateway/Sync as a Conditional Capability

When `connectivity_plus` detects internet, and battery is above threshold, the `GatewayManager` activates. It:

- runs priority-based sync queue
- uploads P1 bundles first (transactions)
- uploads P2 bundles (messages, search requests)
- uploads P3 bundles (delivery logs)
- uploads P4 bundles (media, web snapshots) last

9. Server Communication

Use `Dio` for HTTP communication with the central server. The server exposes simple REST endpoints. All payloads are JSON-serialized bundles.

10. Cryptographic Identity

On first launch, generate an Ed25519 keypair. Store the private key securely using `flutter_secure_storage`. The public key becomes the node's identity and is embedded in every bundle as the source identifier.

Part 2: Full Library/Plugin List by Module

---

1. UI & Navigation

Library	Purpose	pub.dev
`flutter_riverpod`	State management across all features	Yes
`go_router`	Declarative routing between screens	Yes
`flutter_screenutil`	Responsive UI sizing	Yes
`google_fonts`	Typography	Yes
`shimmer`	Loading skeleton animations	Yes
`cached_network_image`	Cache any network images for offline display	Yes

2. Local Storage / Persistence

Library	Purpose
`isar`	Fast embedded NoSQL DB for bundle store and search index. Works on Andr
`drift`	Type-safe SQLite ORM for structured feature data like messages, wallet histo
`hive_flutter`	Alternative to Isar if you want simpler box-based storage
`flutter_secure_storage`	Securely store private keys and sensitive credentials. Uses Keystore on Andro
`shared_preferences`	Small settings like node config, gateway toggle, bundle TTL config
`path_provider`	Get file system paths for storing binary content blocks

3. DTN Core / Bundle System

These are all Dart implementations, no external plugins needed for most.

Library	Purpose
`uuid`	Generate unique BundleIDs (UUID v4)
`isar` or `hive`	Bundle Store persistence (see above)
`collection`	Dart utility for priority queues used in routing
`rxdart`	Reactive streams for bundle event propagation across layers
`freezed`	Immutable bundle/event data models
`json_serializable` + `json_annotation`	Bundle JSON serialization/deserialization
`messagepack_dart`	Efficient binary serialization for bundles (smaller than JSON,

4. Peer Discovery (Contact Manager)

Library	Purpose
`bonsoir`	mDNS/DNS-SD for local peer discovery. Works on Android and Windows. Nodes re
`network_info_plus`	Get current Wi-Fi SSID, gateway IP, local IP address
`dart:io`	Raw network access built in

5. Transport Layer (P2P over LAN)

Library	Purpose
`dart:io` (built-in)	`ServerSocket` and `Socket` for raw TCP peer communication over Wi-Fi
`stream_channel`	Clean bidirectional stream wrapper over TCP connections
`flutter_p2p_connection`	Wi-Fi Direct (peer-to-peer without router) on Android. Useful for true offline di
`nearby_connections`	Google Nearby Connections API for Android. Supports BLE, Wi-Fi, hotspot. M

Note: For Windows, LAN TCP is sufficient. Wi-Fi Direct is Android only.

---

6. Bluetooth Transport (Optional)

Library	Purpose
`flutter_blue_plus`	BLE scanning, connecting, reading, writing. Better maintained than original
`flutter_bluetooth_serial`	Classic Bluetooth (SPP) for higher-throughput connections on Android

---

7. Data Layer (Content-Addressed Storage)

Since full IPFS is not embeddable in Flutter directly, implement a lightweight CID system in Dart:

Library	Purpose
`crypto` (built-in Flutter)	SHA-256 hashing of file blocks
`pointycastle`	Broader crypto including Merkle hash trees if needed
`path_provider`	Store content blocks on device filesystem
`convert`	Base58/Base32 encoding for CID representation
`chunked_stream`	Efficiently chunk large files into blocks for processing
`file_saver`	Save reconstructed files to user storage

If you want to actually talk to a real IPFS node (on server side or companion process):

Library	Purpose
`ipfs_http_rpc`	HTTP RPC client to talk to a running IPFS daemon. Useful if you have a companion G...

8. Cryptographic Identity & Signing

Library	Purpose
`cryptography`	Modern Dart crypto. Supports Ed25519 (signing), AES-GCM (encryption), SHA
`pointycastle`	Alternative with broader algorithm support
`flutter_secure_storage`	Store private keys safely on device
`convert`	Hex/Base64 encoding for keys and signatures
`uuid`	Node identity generation support

9. Wallet / Payment System

All wallet logic is implemented in Dart. The core is an append-only transaction log in your local DB.

Library	Purpose
`cryptography`	Sign SpendEvents with Ed25519
`drift`	Store wallet transaction log in structured SQLite
`qr_flutter`	Generate QR codes for wallet addresses / payment requests
`mobile_scanner`	Scan QR codes to receive payment address
`decimal`	Precise decimal arithmetic for token amounts

10. File & Media Sharing

Library	Purpose
`file_picker`	Pick any file type from device
`image_picker`	Pick images and videos specifically
`path_provider`	Get correct storage directories
`open_file`	Open received files with system viewer
`mime`	Detect file MIME type from bytes/extension
`photo_view`	Zoomable image viewer for received media
`video_player`	Play received video files
`flutter_cache_manager`	Manage cached media files
`chunked_stream`	Stream-based chunking for large file processing

11. E-Commerce Feature

Library	Purpose
`drift`	Store product listings, orders, order status locally
`cached_network_image`	If product images ever come from server
`flutter_slidable`	Swipeable list tiles for order management
`intl`	Currency/date formatting
`decimal`	Precise token/price arithmetic

12. Offline Web Search & Browser

Library	Purpose
`flutter_inappwebview`	Render cached HTML snapshots as offline web pages. This is the best webview
`isar` or `drift`	Store local search index (keyword to metadata mapping)
`html`	Parse HTML content if you need to extract keywords for local indexing
`string_similarity`	Fuzzy keyword matching for search results

13. Connectivity & Gateway Detection

Library	Purpose
`connectivity_plus`	Detect network type: none, Wi-Fi, mobile data
`internet_connection_checker`	Actually verify real internet reachability by pinging an external host (cor
`battery_plus`	Check battery level before enabling gateway mode (per your Chapter 3

14. Background Service & Processing

This is critical for DTN to work properly.

Library	Purpose
`flutter_background_service`	Long-running background service for peer discovery loop, forwarder loo
`workmanager`	Periodic background tasks for deferred sync on Android
`flutter_isolate`	Run heavy DTN core processing in separate Dart isolate to avoid UI threa

15. Synchronization Engine / Server API Client

Library	Purpose
`dio`	HTTP client for server sync API calls. Supports interceptors, retry, cancellation whic
`pretty_dio_logger`	Debug logging of sync requests
`retrofit`	Type-safe REST API client generated from annotations. Works with dio

16. Notifications

Library	Purpose
`flutter_local_notifications`	Show local notifications for received messages, payment confirmations,
`awesome_notifications`	Richer notification styles if needed

17. Serialization

Library	Purpose
`freezed`	Immutable data classes for all models. Reduces boilerplate
`json_serializable`	Auto-generate JSON conversion for models
`messagepack_dart`	Binary serialization for efficient bundle transmission over TCP
`protobuf` (optional)	Protocol Buffers. More efficient than JSON for bundle serialization but more setup

18. Testing

Library	Purpose
`flutter_test`	Built-in Flutter testing
`mocktail`	Mocking for unit tests
`integration_test`	End-to-end integration tests
`faker_dart`	Generate fake data for tests

19. Development & Debugging

Library	Purpose
`logger`	Structured logging throughout the app for debugging DTN behavior
`flutter_dotenv`	Manage server URLs and config per environment
`package_info_plus`	App version info

Summary Table: Libraries by Layer

Layer	Key Libraries
UI	riverpod, go_router, flutter_screenutil
Bundle Model & Serialization	freezed, json_serializable, messagepack_dart, uuid
Bundle Store	isar, hive
Feature Data	drift
Identity & Crypto	cryptography, flutter_secure_storage
Peer Discovery	bonsoir, network_info_plus

Layer	Key Libraries
P2P Transport	dart:io, flutter_p2p_connection, nearby_connections
Bluetooth	flutter_blue_plus
Content Storage	path_provider, crypto, convert, chunked_stream
Wallet	cryptography, qr_flutter, mobile_scanner, decimal
File/Media	file_picker, image_picker, open_file, video_player
Commerce	drift, intl, decimal
Offline Browser	flutter_inappwebview, html
Connectivity	connectivity_plus, internet_connection_checker, battery_plus

---

Critical Observations Before You Start

1. The hardest parts in Flutter are:

- Background peer discovery and forwarding on Android (battery optimization will kill your service without foreground notification)
- Wi-Fi Direct on newer Android versions has changed permissions
- Windows has no `nearby_connections` support so you rely purely on LAN TCP + mDNS for Windows nodes

2. The easiest parts:

- All crypto, bundle logic, routing algorithms are pure Dart
- Local storage is well-supported
- HTTP server sync is straightforward with Dio

3. You will likely need two companion pieces outside Flutter:

- A simple **Go or Node.js central server** for reconciliation, web scraping, and content mirroring

- Possibly a **Go sidecar** on desktop (Windows/Linux) for real IPFS if you want full content-addressing beyond your lightweight CID implementation

---

Do you want me to now go into any of these specific areas in more detail? For example we could:

1. Design the DTN Core module in detail with actual Dart code structure
2. Design the wallet/payment system implementation
3. Plan your MVP build order (what to build first for the demo)
4. Design the central server architecture